DevGuardian

AI-Native CI/CD Quality Gate for Azure DevOps

Microsoft Hackathon — Theme 06: Production Function

Zero-cost build · NVIDIA NIM free models · Azure DevOps free APIs

1. Project overview

DevGuardian is an AI-native quality gate that plugs into Azure DevOps pipelines. Unlike generic AI code reviewers, it introduces two unique innovations no other hackathon team will have: a Developer Trust Score (DTS) that routes each PR to the correct depth of AI scrutiny, and a Codebase DNA Fingerprint that learns your team's specific patterns and flags violations of YOUR standards — not generic rules.

Theme	06 — Production Function (AI-Powered Production)
Core problem	PRs wait 2-3 days for human review; security bugs reach production; no intelligence between 'ready
Unique angle	Developer Trust Score + Codebase DNA Fingerprint — stateful, per-developer intelligence that learn
Tech cost	Rs 0 — all NVIDIA NIM free tier models + open-source tools + Azure DevOps free APIs
Backend	Python + FastAPI hosted on Railway (free tier)
Database	SQLite (DTS store) + ChromaDB (vector store for DNA fingerprint)

2. What makes this unique vs every other AI submission

Most teams will build: 'AI reviews code and shows a list of issues.' DevGuardian is fundamentally different in three ways:

What 90% of teams will build	What DevGuardian does differently
Same depth of review for every PR regardless of developer history	DTS-adaptive review: high-trust devs get 8-sec shallow scan; low-trust get deep 45-sec audit
Stateless — learns nothing across PRs, no memory of past behaviour	Stateful — builds per-developer knowledge graph over months, improves continuously
Checks code against generic Stack Overflow / lint rules	DNA Fingerprint checks against YOUR codebase history — enforces YOUR team's actual standards
Catches individual bugs — misses team-level systemic patterns	Team Risk Intelligence Report: 'These 3 devs consistently introduce auth bugs — suggest pairing'

3. Developer Trust Score (DTS) — the core innovation

Every developer gets a DTS from 0–100, recalculated weekly from their Azure DevOps history. The score is NEVER shown to managers or HR. It is used only by the AI pipeline to decide how deeply to inspect each PR — high trust means faster feedback, low trust means deeper protection.

3.1 The 5 signals

Signal	What it measures	Weight	API endpoint
Bug introduction rate	Bugs per 10 merged PRs (linked work items tagged Bug, created after merge date)	**-20 pts**	`/pullRequests + /workitems`
PR revert rate	% of merged PRs reverted within 7 days (commit message starts with 'Revert')	**-15 pts**	`/commits`
Test coverage delta	Avg % test coverage change across developer's PRs — positive means adding tests	**+0.3/%**	`/test/codecoverage`
Security findings count	Count of HIGH/CRITICAL findings in developer's PRs in last 90 days	**-10 pts**	`/annotations`
First-pass accept rate	% of PRs approved without any change-request thread — measures submission quality	**+5 pts/%**	`/pullRequests/threads`

3.2 The formula

```
DTS = 70 # neutral baseline DTS -= bug_rate x 20 # bugs/10 PRs: 0 = no penalty, 2 = -40 DTS -= revert_rate x 15 # 0.0-1.0: 100% reverts = -15 DTS += coverage_delta x 0.3 # -10 to +15%: +10% coverage = +3 DTS -= security_count x 10 # per finding: 2 issues = -20 DTS += accept_rate x 5 # 0.0-1.0: 100% first-pass = +5 DTS = clamp(DTS, 0, 100) # never below 0 or above 100
```

3.3 DTS to review depth routing

DTS range	Depth	AI model used	What happens
80–100	**Shallow**	Phi-3-medium (NVIDIA NIM)	Quick scan only, ~8 sec
50–79	**Standard**	Llama 3.1-70b (NVIDIA NIM)	Full review, ~25 sec
20–49	**Deep**	Codestral + Llama 3.1 parallel	Security + test gen, ~45 sec
0–19	**Block PR**	No merge allowed	Team lead notified on Teams

4. The 6 core modules

Each module is independent, can be built and tested separately, and communicates via a shared SQLite database and a central FastAPI orchestrator.

#	Module	What it does	Free tool used
1	**DTS Engine**	Calculates Developer Trust Score from 5 Azure DevOps signals. Routes PR to correct review depth.	`Python + SQLite`

2	**DNA Fingerprint Engine**	Embeds 6 months of merged PRs into ChromaDB. Detects style violations specific to YOUR codebase, not generic rules.	`NVIDIA embed-qa-4 + ChromaDB`
3	**Security Scanner**	Runs Semgrep OSS static analysis on PR diff. Sends findings to Llama 3.1 for plain-English explanation + fix.	`Semgrep OSS + NVIDIA NIM`
4	**Test Gap Detector**	Python AST finds untested functions in PR. Codestral generates missing pytest/jest tests and commits them back.	`mistralai/codestral-22b (NIM)`
5	**Team Risk Intelligence**	Weekly aggregation of DTS data. Llama 3.1 writes anonymised team-level risk report posted to Teams channel.	`Llama 3.1 + Teams Webhook`
6	**Deployment Risk Gate**	Aggregates DTS of all authors in a release + unresolved findings. Red/Yellow/Green gate blocks risky deploys.	`Azure Pipelines API + Teams`

5. System architecture — 3-layer flow

Layer	What it does	Technology
Layer 1 Event trigger	Azure DevOps webhook fires on every PR created or updated. POSTs full PR payload to FastAPI /webhook endpoint.	`Azure DevOps Webhooks (free)`
Layer 2A Context builder	Fetches PR diff, author DTS from SQLite, linked work items, and past incident records for changed files.	`Azure DevOps REST APIs`
Layer 2B DTS engine	Reads developer's DTS from SQLite. Decides review depth: Shallow / Standard / Deep / Block.	`Python + SQLite`
Layer 2C AI review orchestrator	Routes to correct NVIDIA NIM model based on depth. Shallow uses Phi-3. Deep uses Codestral + Llama 3.1 in parallel.	`NVIDIA NIM free tier`
Layer 2D DNA fingerprint check	Embeds PR code chunks. Queries ChromaDB for style violations specific to this codebase.	`NVIDIA embed-qa-4 + ChromaDB`
Layer 3 Action engine	Posts inline PR comments, updates DTS in SQLite, sends Teams alert for HIGH risk, blocks merge if DTS < 20.	`Azure DevOps API + Teams Webhook`

6. Complete free technology stack — total cost Rs 0

Every component uses either NVIDIA NIM free tier (access via build.nvidia.com with a free account), open-source Python libraries, or free-tier cloud services. No credit card required for any of these.

Tool / Model	Used for	How to get it free
`NVIDIA NIM — Llama 3.1-70b`	Code review, summaries, standup briefs, risk reports	build.nvidia.com — free credits
`NVIDIA NIM — Codestral-22b`	Test generation, code rewriting, PR analysis	build.nvidia.com — free credits

`NVIDIA NIM — Phi-3-medium`	Shallow (fast) review for high-DTS developers	build.nvidia.com — free credits
`NVIDIA embed-qa-4`	Embeddings for DNA fingerprint engine	build.nvidia.com — free credits
`ChromaDB`	Local vector database for codebase DNA storage	pip install chromadb
`Semgrep OSS`	Static analysis — 500+ security rules, no API key	pip install semgrep
`Azure DevOps free tier`	All pipeline, PR, boards, and test APIs	Personal account — free
`SQLite`	Developer stats and DTS history storage	Built into Python — no install
`Railway / Render`	Host FastAPI backend	Free tier — no credit card
`Teams Incoming Webhook`	Post alerts and reports to Teams channel	Free URL from Teams settings
`GitHub Actions`	Cron jobs, automation triggers	Free 2000 min/month

7. Key code reference

7.1 NVIDIA NIM API call — adaptive by DTS depth

```
import requests def review_pr(diff: str, dts_score: int) -> dict: depth = "shallow" if dts_score > 80 else \ "standard" if dts_score > 50 else "deep" prompts = { "shallow": "Quick scan: list only critical bugs.", "standard": "Full review: bugs, logic errors, improvements.", "deep": "Deep audit: security, logic, test gaps, architecture." } response = requests.post( "https://integrate.api.nvidia.com/v1/chat/completions", headers={"Authorization": f"Bearer {NVIDIA_API_KEY}"}, json={ "model": "meta/llama-3.1-70b-instruct", "messages": [ {"role": "system", "content": prompts[depth]}, {"role": "user", "content": diff} ], "response_format": {"type": "json_object"} } ) return response.json()["choices"][0]["message"]["content"]
```

7.2 DTS calculation function

```
def calculate_dts(bug_rate, revert_rate, coverage_delta, security_count, accept_rate) -> dict: score = 70 score -= bug_rate * 20 # bugs per 10 PRs score -= revert_rate * 15 # ratio 0.0-1.0 score += coverage_delta * 0.3 # % change in coverage score -= security_count * 10 # count of HIGH/CRITICAL findings score += accept_rate * 5 # ratio 0.0-1.0 dts = max(0, min(100, round(score))) depth = ("shallow" if dts >= 80 else "standard" if dts >= 50 else "deep" if dts >= 20 else "block") return {"dts": dts, "depth": depth}
```

8. 5-day hackathon build timeline

Day	Task
Day 1	Setup FastAPI skeleton. Register Azure DevOps free account. Create test repo. Configure PR webhook — confirm payload received.
Day 1-2	Build DTS Engine with seeded mock data for 5 developers. Confirm routing logic (Shallow/Standard/Deep/Block) works correctly.

Day 2	Sign up at build.nvidia.com, get free API key. Connect NVIDIA NIM. Test all 3 review prompt templates with sample PR diffs.
Day 3	Build DNA Fingerprint Engine: embed 20-30 past PRs into ChromaDB using embed-qa-4. Test style violation detection on new PR.
Day 3-4	Add Security Scanner (Semgrep OSS subprocess) and Test Generator (Codestral via NIM). Post inline PR comments via Azure DevOps API.
Day 5	Build Team Risk Report + Deployment Gate. Set up Teams webhook. Record demo video: PR submitted → 45s → inline comments appear + Teams alert fires.

9. Judge pitch — key talking points

> *"Every team here built an AI that reviews code. We built an AI that **knows your developers**. DevGuardian does not just catch bugs — it understands that some developers consistently introduce auth vulnerabilities, that others always ship clean code, and that your payments module has a 3x higher bug rate than your auth module. It uses that intelligence to protect Microsoft's production systems before a single line of bad code is merged."*

Demo flow for maximum judge impact:

1. **Show the DTS dashboard (30 sec)**
2. **Submit a PR with planted bugs live (60 sec)**
3. **Submit same PR from high-DTS developer (20 sec)**
4. **Show the Team Risk Intelligence Report (20 sec)**

Open the web UI showin
explain every PR from th

Submit a PR from the lo
buggy lines in Azure Dev

Same PR from a trusted
in 10 seconds. Show the

Weekly Teams message
the last 6 months. Recor

10. Microsoft business value

Microsoft product / team	How DevGuardian helps them
Azure DevOps (10M+ users)	DevGuardian plugs directly into Azure DevOps. Every PR review saved = engineering hours reclaimed across all Microsoft product teams.
Windows / Office releases	Flaky tests and security bugs delay major releases. DTS-based deep review on low-trust PRs catches these before they enter the release branch.
GitHub Copilot ecosystem	DevGuardian complements Copilot — Copilot writes code, DevGuardian guards what was written. Natural extension of Microsoft's AI dev story.
Azure SLA protection	The Deployment Risk Gate prevents risky releases from reaching Azure production — directly protecting Microsoft's 99.9% SLA commitments.

This document is a complete project brief for DevGuardian — Microsoft Hackathon Theme 06 submission. Upload this PDF to any Claude conversation to instantly restore full project context and continue building from where you left off. All code, APIs, formulas, and architecture are self-contained in this brief.